Author

Name: Mohit
Roll Number: 24f2001273
Email: 24f2001273@ds.study.iitm.ac.in
About Me: I am a student in the IIT Madras BS Degree program with a strong interest in full-stack web development and scalable architectures. I enjoy building applications that solve real-world logistical problems by combining robust backend logic with clean, responsive user interfaces.

AI/LLM usage

I used Gemini to assist me in architectural planning, debugging Flask logic errors (such as resolving database integrity constraints), generating some modal codes for Vue.js components, and some styling. The extent of AI/LLM usage is around **15–20%**. All core business logic specifically, the double-booking prevention algorithms, Celery background job integration, and state management was thoroughly reviewed, modified, and manually integrated by me to fit the specific requirements of this project's rubric.

Description

This project is a comprehensive, role-based Hospital Management System (HMS) designed to streamline hospital operations. It allows patients to search for doctors and securely book appointments, enables doctors to manage their availability and log medical treatments, and provides administrators with oversight tools to manage system users.

Technologies used

- **Flask & Flask-RESTful:** Core backend web framework used to build scalable REST APIs.
- **Vue.js 3:** Frontend framework used to build a fast, responsive Single Page Application (SPA) dashboard.
- **Flask-SQLAlchemy:** Object Relational Mapper (ORM) used to manage the SQLite database efficiently.
- **Flask-JWT-Extended:** Used for managing secure user authentication and Role-Based Access Control (RBAC).
- **Celery & Redis:** Used as an asynchronous task queue and message broker to handle background jobs (like scheduled reminders and CSV/PDF report generation) without freezing the main application.
- **Bootstrap 5:** Used for frontend UI styling, modal generation, and ensuring mobile responsiveness.

DB Schema Design

The database is built on SQLite using SQLAlchemy.

- **User:** Base authentication table (username, password, role, is_blacklisted).
- **Role Profiles (Admin, Doctor, Patient):** Linked to the User table via a 1-to-1 relationship (`user_id` Foreign Key). Doctors include `department_id` and `doctor_fees`.
- **Department:** Stores hospital specializations (name, description).
- **DoctorAvailability:** Tracks specific dates and time slots for doctors.
- **Appointment:** Links Patients to Doctors (patient_id, doctor_id, date, time, status). Includes constraints to prevent double-booking.
- **Treatment:** Linked 1-to-1 with Appointments to store medical notes and prescriptions.
- **Reasoning:** The schema uses a decoupled User-to-Profile design. This ensures that authentication logic remains unified in one table, while role-specific data (like a doctor's fees or a patient's age) remains isolated in separate tables to prevent null-heavy, bloated database rows.

API Design

The backend API is RESTful and heavily secured using JWT Bearer tokens. APIs are grouped by system roles:

- **Auth APIs:** Handles `/api/login` and `/api/register` to issue access tokens.
- **Public APIs:** Unprotected routes for fetching hospital Departments and Doctor lists (optimized with Redis caching).
- **Role-Protected APIs:** Routes prefixed with `/api/admin/`, `/api/doctor/`, and `/api/patient/`. These endpoints enforce strict role checks to ensure, for example, that only patients can hit the `/api/patient/book` endpoint to schedule a slot.

Architecture and Features

The project is organized into a decoupled frontend and backend architecture. The backend (`/backend/`) houses `app.py` containing the main Flask initialization, Celery worker configurations, and API routing. Database models are isolated in `models.py`. The frontend (`/frontend/`) is a Vue.js application where components are organized inside `src/views/` (separated by Admin, Doctor, and Patient dashboards) and routed securely using `src/router/`.

Key Features Implemented:

- **Strict RBAC:** Role-Based Access Control ensuring secure routing on both the Vue frontend and Flask backend.
- **Dynamic Booking Engine:** Real-time appointment scheduling that automatically prevents double-booking of time slots.

- **API Caching:** High-traffic endpoints, such as the Admin Dashboard statistics, are cached using Redis to minimize database load.
- **Asynchronous Background Jobs:** Celery is utilized to process long-running tasks, specifically user-triggered CSV history exports and monthly PDF generation, alerting the user upon completion.
- **UI/UX Enhancements:** A fully mobile-responsive, dark-themed "glassmorphic" user interface.

Video

https://drive.google.com/file/d/19ipkbe2sznxYOkPwx-AF8DBTfIxZAqR5/view?usp=sharing